

UNIT-III

3.5 Database Normalization

Normalization is an important process in database design that helps improve the database's efficiency, consistency, and accuracy. It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs.

- Database normalization is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places).
- Data redundancy unnecessarily increases the size of the database as the same data is repeated in many places. Inconsistency problems also arise during insert, delete, and update operations.
- In the relational model, there exist standard methods to quantify how efficient a database is. These methods are called [normal forms](#) and there are algorithms to convert a given database into normal forms.
- Normalization generally involves splitting a table into multiple ones which must be linked each time a query is made requiring data from the split tables.

Why do we need Normalization?

The primary objective for normalizing the relations is to eliminate the below anomalies. Failure to reduce anomalies results in data redundancy, which may threaten data integrity and cause additional issues as the database increases. Normalization consists of a set of procedures that assist you in developing an effective database structure.

- **Insertion Anomalies:** Insertion anomalies occur when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete. For example, if a database requires that every record has a primary key, but no value is provided for a particular record, it cannot be inserted into the database.
- **Deletion anomalies:** Deletion anomalies occur when deleting a record from a database and can result in the unintentional loss of data. For example, if a database contains information about customers and orders, deleting a customer record may also delete all the orders associated with that customer.
- **Update anomalies:** Update anomalies occur when modifying data in a database and can result in inconsistencies or errors. For example, if a database contains information about employees and their salaries, updating an employee's salary in one record but not in all related records could lead to incorrect calculations and reporting.

Before Normalization

Employee_Department

Emp_ID	Emp_Name	Department	Dept_Location	Emp_Skills
101	Nick Wise	HR	London	Recruitment, Payroll
102	John Cader	Finance	Australia	Budgeting
103	Lily Case	HR	London	Recruitment
104	Ford Dawid	IT	Chicago	Programming, Testing

After Normalization

Employee

Emp_ID	Emp_Name	Dept_ID
101	Nick Wise	D1
102	John Cader	D2
103	Lily Case	D1
104	Ford Dawid	D3

Department

Dept_ID	Department	Dept_Location
D1	HR	London
D2	Finance	Australia
D3	IT	Chicago

Employee_Skills

Emp_ID	Emp_Skills
101	Recruitment
101	Payroll
102	Budgeting
103	Recruitment
104	Programming
104	Testing

Before Normalization: The table is prone to redundancy and anomalies (insertion, update, and deletion).

After Normalization: The data is divided into logical tables to ensure consistency, avoid redundancy and remove anomalies making the database efficient and reliable.

Features of Database Normalization

- **Elimination of Data Redundancy:** One of the main features of normalization is to eliminate the data redundancy that can occur in a database. Data redundancy refers to the repetition of data in different parts of the database. Normalization helps in reducing or eliminating this redundancy, which can improve the efficiency and consistency of the database.
- **Ensuring Data Consistency:** Normalization helps in ensuring that the data in the database is consistent and accurate. By eliminating redundancy, normalization helps in preventing inconsistencies and contradictions that can arise due to different versions of the same data.
- **Simplification of Data Management:** Normalization simplifies the process of managing data in a database. By breaking down a complex data structure into simpler tables, normalization makes it easier to manage the data, update it, and retrieve it.
- **Improved Database Design:** Normalization helps in improving the overall design of the database. By organizing the data in a structured and systematic way, normalization makes it easier to design and maintain the database. It also makes the database more flexible and adaptable to changing business needs.
- **Avoiding Update Anomalies:** Normalization helps in avoiding update anomalies, which can occur when updating a single record in a table affects multiple records in other tables. Normalization ensures that each table contains only one type of data and that the relationships between the tables are clearly defined, which helps in avoiding such anomalies.
- **Standardization:** Normalization helps in standardizing the data in the database. By organizing the data into tables and defining relationships between them, normalization helps in ensuring that the data is stored in a consistent and uniform manner.

Normal Forms in DBMS

Normal Forms	Description of Normal Forms
First Normal Form (1NF)	A relation is in first normal form if every attribute in that relation is single-valued attribute.

Normal Forms	Description of Normal Forms
Second Normal Form (2NF)	A relation that is in First Normal Form and every non-primary-key attribute is fully functionally dependent on the primary key, then the relation is in Second Normal Form (2NF) .
Third Normal Form (3NF)	A relation is in the third normal form, if there is no transitive dependency for non-prime attributes as well as it is in the second normal form. A relation is in 3NF if at least one of the following conditions holds in every non-trivial function dependency $X \rightarrow Y$. • X is a super key. • Y is a prime attribute (each element of Y is part of some candidate key).
Boyce-Codd Normal Form (BCNF)	For BCNF the relation should satisfy the below conditions • The relation should be in the 3rd Normal Form. • X should be a super-key for every functional dependency (FD) $X \rightarrow Y$ in a given relation.
Fourth Normal Form (4NF)	A relation R is in 4NF if and only if the following conditions are satisfied: • It should be in the Boyce-Codd Normal Form (BCNF). • The table should not have any Multi-valued Dependency.
Fifth Normal Form (5NF)	A relation R is in 5NF if and only if it satisfies the following conditions: • R should be already in 4NF. • It cannot be further non loss decomposed (join dependency).

Advantages of Normalization

- Normalization eliminates data redundancy and ensures that each piece of data is stored in only one place, reducing the risk of data inconsistency and making it easier to maintain data accuracy.
- By breaking down data into smaller, more specific tables, normalization helps ensure that each table stores only relevant data, which improves the overall data integrity of the database.
- Normalization simplifies the process of updating data, as it only needs to be changed in one place rather than in multiple places throughout the database.
- Normalization enables users to query the database using a variety of different criteria, as the data is organized into smaller, more specific tables that can be joined together as needed.
- Normalization can help ensure that data is consistent across different applications that use the same database, making it easier to integrate different applications and ensuring that all users have access to accurate and consistent data.

Disadvantages of Normalization

- Normalization can result in increased performance overhead due to the need for additional [join operations](#) and the potential for slower query execution times.
- Normalization can result in the loss of data context, as data may be split across multiple tables and require additional joins to retrieve.
- Proper implementation of normalization requires expert knowledge of database design and the normalization process.
- Normalization can increase the complexity of a database design, especially if the data model is not well understood or if the normalization process is not carried out correctly.

Normal Forms in DBMS

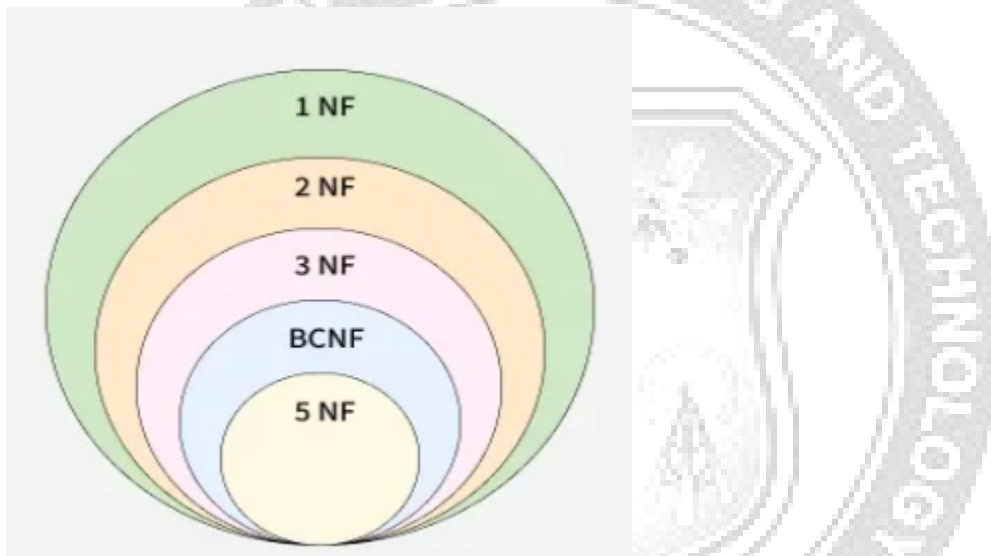
Normal forms are a set of progressive rules (or design checkpoints) for relational schemas that reduce redundancy and prevent data anomalies. Each normal form - 1NF, 2NF, 3NF, BCNF, 4NF, 5NF - is stricter than the previous one: meeting a higher normal form implies the lower ones are satisfied. Think of them as layers of cleanliness for your tables: the deeper you go, the fewer redundancy and integrity problems you'll have.

Benefits of using Normal Forms:

- Reduce duplicate data and wasted storage.

- Prevent insert, update, and delete anomalies.
- Improve data consistency and integrity.
- Make the schema easier to maintain and evolve.

The Diagram below shows the hierarchy of database normal forms. Each inner circle represents a stricter level of normalization, starting from 1NF (basic structure) to 5NF (most refined). As you move inward, data redundancy reduces and data integrity improves. Each level builds upon the previous one to ensure a cleaner and more efficient database design.



1. First Normal Form (1NF): Eliminating Duplicate Records

A table is in [1NF](#) if it satisfies the following conditions:

- All columns contain atomic values (i.e., indivisible values).
- Each row is unique (i.e., no duplicate rows).
- Each column has a unique name.
- The order in which data is stored does not matter.

Example of 1NF Violation: If a table has a column "Phone Numbers" that stores multiple phone numbers in a single cell, it violates 1NF. To bring it into 1NF, you need to separate phone numbers into individual rows.

2. Second Normal Form (2NF): Eliminating Partial Dependency

A relation is in [2NF](#) if it satisfies the conditions of 1NF and additionally. No partial dependency exists, meaning every non-prime attribute (non-key attribute) must depend on the entire primary key, not just a part of it.

Example: For a composite key (StudentID, CourseID), if the "StudentName" depends only on "StudentID" and not on the entire key, it violates 2NF. To normalize, move StudentName into a separate table where it depends only on "StudentID".

3. Third Normal Form (3NF): Eliminating Transitive Dependency

A relation is in [3NF](#) if it satisfies 2NF and additionally, there are no [transitive dependencies](#). In simpler terms, non-prime attributes should not depend on other non-prime attributes.

Example: Consider a table with (StudentID, CourseID, Instructor). If Instructor depends on "CourseID", and "CourseID" depends on "StudentID", then Instructor indirectly depends on "StudentID", which violates 3NF. To resolve this, place Instructor in a separate table linked by "CourseID".

4. Boyce-Codd Normal Form (BCNF): The Strongest Form of 3NF

[BCNF](#) is a stricter version of 3NF where for every non-trivial functional dependency ($X \rightarrow Y$), X must be a [superkey](#) (a unique identifier for a record in the table).

Example: If a table has a dependency (StudentID, CourseID) $\rightarrow$ Instructor, but neither "StudentID" nor "CourseID" is a superkey, then it violates BCNF. To bring it into BCNF, decompose the table so that each determinant is a candidate key.

5. Fourth Normal Form (4NF): Removing Multi-Valued Dependencies

A table is in [4NF](#) if it is in BCNF and has no multi-valued dependencies. A multi-valued dependency occurs when one attribute determines another, and both attributes are independent of all other attributes in the table.

Example: Consider a table where (StudentID, Language, Hobby) are attributes. If a student can have multiple hobbies and languages, a multi-valued dependency exists. To resolve this, split the table into separate tables for Languages and Hobbies.

6. Fifth Normal Form (5NF): Eliminating Join Dependency

[5NF](#) is achieved when a table is in 4NF and all join dependencies are removed. This form ensures that every table is fully decomposed into smaller tables that are logically connected without losing information.

Example: If a table contains (StudentID, Course, Instructor) and there is a dependency where all combinations of these columns are needed for a specific relationship, you would split them into smaller tables to remove redundancy.

Applications of Normal Forms in DBMS

- **Ensures Data Consistency:** Prevents data anomalies by ensuring each piece of data is stored in one place, reducing inconsistencies.
- **Reduces Data Redundancy:** Minimizes repetitive data, saving storage space and avoiding errors in data updates or deletions.
- **Improves Query Performance:** Simplifies queries by breaking large tables into smaller, more manageable ones, leading to faster data retrieval.
- **Enhances Data Integrity:** Ensures that data is accurate and reliable by adhering to defined relationships and constraints between tables.
- **Easier Database Maintenance:** Simplifies updates, deletions, and modifications by ensuring that changes only need to be made in one place, reducing the risk of errors.
- **Facilitates Scalability:** Makes it easier to modify, expand, or scale the database structure as business requirements grow.
- **Supports Better Data Modeling:** Helps in designing databases that are logically structured, with clear relationships between tables, making it easier to understand and manage.
- **Reduces Update Anomalies:** Prevents issues like insertion, deletion, or modification anomalies that can arise from redundant data.
- **Improves Data Integrity and Security:** By reducing unnecessary data duplication, normal forms help ensure sensitive information is securely and correctly maintained.
- **Optimizes Storage Efficiency:** By organizing data into smaller tables, storage is used more efficiently, reducing the overhead for large databases.